Java

3.5. Продолжение основ языка

Исключения, дженерики и практика



Продолжаем основы Java

- Исключения: как прерывается выполнение и где обрабатывать ошибку
- Дженерики: один класс или метод для разных типов
- Явный аргумент типа: Collections.<Cat>emptyList()
- Практика: собираем небольшой список задач

От обработки ошибки до собственного Box<T>.

Исключение прерывает обычный ход

Фрагмент внутри main

```
System.out.println("Начало");
int value = Integer.parseInt("abc");
System.out.println("Конец");
```

Строка "abc"
не является
целым числом.

Возникает
NumberFormatException.

«Конец» не выведется, если исключение никто не обработал.

try и catch

Фрагмент внутри main

```
String text = "abc";

try {
    int value = Integer.parseInt(text);
    System.out.println(value);
} catch (NumberFormatException error) {
    System.out.println("Введите целое число");
}
System.out.println("Продолжаем");
```

try содержит операцию, которая может не удаться.

catch обрабатывает конкретный тип исключения.

Не оставляем пустой catch и не скрываем причину ошибки.

finally: действие при выходе из try

Фрагмент внутри main

```
try {
    System.out.println("Пробуем");
    Integer.parseInt("abc");
} catch (NumberFormatException error) {
    System.out.println("Ошибка числа");
} finally {
    System.out.println("Завершаем");
}
System.out.println("Дальше");
```

Порядок вывода:
Пробуем
Ошибка числа
Завершаем
Дальше

finally выполняется и при нормальном выходе, и при исключении.

throw: запрещаем некорректную задачу

Конструктор внутри Task с полем private String title

```
Task(String title) {
    if (title == null || title.isBlank()) {
        throw new IllegalArgumentException(
            "Название не должно быть пустым"
        );
    }
    this.title = title;
}
```

Конструктор проверяет вход.

При ошибке объект не возвращается вызывающему коду.

Если объект создан успешно, у него есть допустимое название.

Checked и unchecked: что проверяет компилятор

Категория	Пример	Требование компилятора
Checked	IOException	Обработать в catch или объявить в throws
Unchecked	IllegalArgumentException NumberFormatException	catch и throws не обязательны

Обе категории могут возникать во время выполнения программы.

throws и checked-исключения

Метод в Main; импорты перечислены в заметках

```
static String readText(Path path)
        throws IOException {
    return Files.readString(path);
}
```

throws сообщает: ошибка может уйти вызывающему коду.

IOException нужно обработать или объявить дальше.

throw бросает исключение, throws описывает его в объявлении.

Ресурсы: try-with-resources

Фрагмент внутри main

```
Path path = Path.of("tasks.txt");
try (BufferedReader reader =
         Files.newBufferedReader(path)) {
    System.out.println(reader.readLine());
} catch (IOException error) {
    System.out.println("Не удалось прочитать файл");
}
```

reader автоматически закроется при выходе из try.

Закрытие произойдёт и при исключении.

GC освобождает память. Файл нужно закрывать своевременно.

Как читать ошибку и пользоваться отладчиком

- Посмотреть тип исключения и сообщение
- Найти ближайшую строку своего кода в стеке вызовов
- Поставить breakpoint перед проблемным действием
- Проверить значения и пройти выполнение по шагам

Сначала воспроизводим проблему, затем меняем код.

Дженерики: тип как параметр

От коробки с котом до List<Task>

Одна коробка, разные типы содержимого

- Для кота нужна коробка, которая возвращает Cat
- Для строки нужна такая же коробка, которая возвращает String
- Хранение и получение устроены одинаково
- Хотим описать класс один раз и сохранить проверку типов

Box<T>: T обозначает тип содержимого коробки.

Объявляем обобщённый класс Box<T>

Файл Box.java

```
class Box<T> {
    private final T value;

    Box(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}
```

<T> объявляет параметр типа.

T используется как тип поля, параметра конструктора и результата get().

T связывает тип того, что кладём, с типом того, что получаем.

Коробка с котом: Box<Cat>

Cat.java: record Cat(String name) {}. Ниже фрагмент внутри main

```
Cat barsik = new Cat("Барсик");
Box<Cat> box = new Box<Cat>(barsik);

Cat cat = box.get();
System.out.println(cat.name());

// Не компилируется:
// new Box<Cat>("не кот");
// String text = box.get();
```

Cat задаёт конкретный тип.

Конструктор принимает Cat.

get() возвращает Cat. Приведение не нужно.

<T> объявляет параметр типа. <Cat> передаёт конкретный тип.

Ромб <> и типы-обёртки

Фрагмент внутри main. Используем тот же класс Box<T>

```
Box<Cat> catBox =
    new Box<>(new Cat("Мурка"));

Box<String> textBox = new Box<>("Java");
Box<Integer> numberBox = new Box<>(42);

System.out.println(textBox.get());
System.out.println(numberBox.get());

// Box<int> не компилируется
```

<> позволяет вывести аргумент типа при создании Box.

В <...> используем ссылочные типы. Для int есть Integer.

new Box<>(...) короче, проверка типов сохраняется.

Обобщённый метод: собственный параметр T

Файл Boxes.java. Box<T> объявлен отдельно

```
class Boxes {
    static <T> T unpack(Box<T> box) {
        return box.get();
    }
}
```

<T> перед типом результата объявляет параметр метода.

Box<T> и результат T связаны одним типом.

Метод может быть обобщённым, даже если сам класс Boxes обычный.

Компилятор выводит T из аргумента

Фрагмент внутри main

```
Box<Cat> catBox =
    new Box<>(new Cat("Барсик"));
Box<String> textBox = new Box<>("Java");

Cat cat = Boxes.unpack(catBox);
String text = Boxes.unpack(textBox);

System.out.println(cat.name());
System.out.println(text);
```

В первом вызове
T соответствует Cat.

Во втором вызове
T соответствует String.

Тело метода одно.

В разных вызовах T может обозначать разные типы.

Метод, который принимает List<Cat>

Метод внутри Main. Импорт java.util.List

```
static void printCats(List<Cat> cats) {
    for (Cat cat : cats) {
        System.out.println(cat.name());
    }
}

// Внутри main:
// printCats(List.of(new Cat("Барсик")));
```

Здесь Cat уже выбран.

Метод принимает список котов.

Собственного параметра типа у метода нет.

List<Cat> в параметрах ещё не делает метод обобщённым.

Аргумент типа можно указать явно

Фрагмент внутри main

```
Box<Cat> box =
    new Box<>(new Cat("Барсик"));

Cat a = Boxes.unpack(box);
Cat b = Boxes.<Cat>unpack(box);

System.out.println(a == b);
```

Оба вызова возвращают Cat.

<Cat> стоит после точки, перед именем метода unpack.

Boxes.<Cat>unpack(box): тип слева от имени метода, после точки.

Collections.<Cat>emptyList()

Фрагмент внутри main. Импорты java.util.List и java.util.Collections

```
List<Cat> a = Collections.emptyList();
List<Cat> b = Collections.<Cat>emptyList();

System.out.println(a.size()); // 0
System.out.println(b.size()); // 0
```

В первой строке тип подсказывает List<Cat> слева.

Во второй <Cat> указан явно перед emptyList.

Collections — класс утилит. Collection — интерфейс коллекции.

var выводит тип, но не отменяет его

Фрагмент внутри main

```
var count = 3;
var title = "Java";

// count = "три"; // ошибка компиляции

System.out.println(count);
System.out.println(title);
```

Компилятор выводит int и String из начальных значений.

Тип переменной не меняется.

В начале обучения явный тип делает пример понятнее.

С var явный аргумент типа уже заметен

Фрагмент внутри main. Импорты java.util.List и java.util.Collections

```
var a = Collections.emptyList();
// Тип a: List<Object>

var b = Collections.<Cat>emptyList();
// Тип b: List<Cat>

List<Cat> cats = b; // OK
// List<Cat> other = a; // ошибка
```

У emptyList() нет аргументов.

Слева var тоже не задаёт Cat.

Без подсказки компилятор выбирает Object.

Последующие строки не меняют тип, который компилятор уже вывел.

Мини-вопрос: что произойдёт?

А разбираем отдельно. B и C выполняем по порядку. Импорты List и Collections

```
// A
Box<String> box = new Box<>("Java");
// Integer value = box.get();

// B
List<Cat> cats = Collections.emptyList();
System.out.println(cats.size());

// C
cats.add(new Cat("Барсик"));
```

A: что будет, если убрать комментарий перед Integer?

B: что выведет?

C: компиляция или ошибка при выполнении?

Сначала определяем типы, затем смотрим на поведение объекта.

Разбор: типы и поведение проверяются по-разному

- A: ошибка компиляции. get() возвращает String, а не Integer
- B: выведет 0. Пустой список существует и имеет размер
- C: компилируется, но add() бросит UnsupportedOperationException
- Для добавления элементов: List<Cat> cats = new ArrayList<>()

List<Cat> описывает тип элементов, но не обещает возможность add().

Практика: собираем приложение

Несколько задач, завершение и вывод результата

Практика: небольшой список задач

- Task хранит название и признак завершения
- Конструктор отвергает null и пустое название
- complete() завершает задачу; isDone() возвращает статус
- В main создайте список из трёх задач и завершите одну
- Выведите названия оставшихся задач и их количество

Проверьте пустой список, все завершённые и повторный complete().

Разбор: поля и конструктор Task

Файл Task.java, первая часть

```
class Task {
    private final String title;
    private boolean done;

    Task(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException(
                "Название не должно быть пустым");
        }
        this.title = title;
    }
    // Методы на следующем слайде
}
```

Название обязательно и не меняется.

done изначально равно false.

Состояние скрыто за private.

Сохраняем класс в Task.java.

Разбор: методы Task

Продолжение Task.java, внутри фигурных скобок класса

```
void complete() {
    done = true;
}

boolean isDone() {
    return done;
}

String getTitle() {
    return title;
}
```

Добавляем методы внутрь класса Task.

Повторный complete оставляет done равным true.

Методы выражают операции, которые нужны вызывающему коду.

Разбор: Main связывает всё вместе

Фрагмент внутри main

```
List<Task> tasks = new ArrayList<>();
tasks.add(new Task("Git"));
tasks.add(new Task("Java"));
tasks.add(new Task("SQL"));
tasks.get(0).complete();

int remaining = 0;
for (Task task : tasks) {
    if (!task.isDone()) {
        System.out.println(task.getTitle());
        remaining++;
    }
}
System.out.println("Осталось: " + remaining);
```

Импорты List и ArrayList.

Результат:
Java
SQL
Осталось: 2

Каждая часть программы отвечает на небольшой понятный вопрос.

Проверяем понимание

- Чем класс отличается от объекта?
- Что хранится в переменной ссылочного типа?
- Чем поле отличается от параметра и локальной переменной?
- Чем return отличается от println и break?
- Что изменится после Task b = a и b.complete()?

Ответьте своими словами и покажите короткий пример.

Что изучать следующим блоком

- Писать тесты для методов и классов
- Уверенно работать с коллекциями и равенством объектов
- Разбирать ошибки и организовывать проект
- После этого переходить к HTTP, JSON и backend-фреймворку

Для первого backend-проекта нужны понятные классы и методы.

Дополнительно: first-class citizens

- Так называют сущности, с которыми можно работать как со значениями
- Например: сохранять, передавать аргументом и возвращать
- В Java так работают со ссылками на объекты
- Ключевое слово class объявляет тип и относится к другой идее

Для базового кода важнее понимать класс, объект и ссылку.

Материалы для повторения

- Dev.java: основы языка, классы, объекты и коллекции
- Java Tutorials: исключения и управление ресурсами
- Java Tutorials: Generics, Generic Methods, Type Inference
- Java API: контракт Collections.emptyList()

Ссылки и пояснения находятся в заметках к слайдам.